

SKILL : WEEK 7

SESSION: 13

Task -1 :

To Parse Input Redirection, Open Files, Redirect stdin, Handle Missing Files, Restore Streams, Test File Inputs

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main()
{
    char input[200];

    printf("Enter command with input redirection\n");
    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    char *redirect = strchr(input, '<');

    if (redirect == NULL)
    {
        printf("No input redirection found.\n");
        return 1;
    }

    *redirect = '\0';

    char *command = input;
    char *filename = redirect + 1;

    while (*filename == ' ')
        filename++;

    while (*command == ' ')
        command++;

    char *end = command + strlen(command) - 1;
    while (end > command && *end == ' ')
    {
        *end = '\0';
        end--;
    }

    while (*filename == ' ')
        filename++;

    int fd = open(filename, O_RDONLY);

    if (fd == -1)

        if (fork() == 0)
        {
            dup2(fd, STDIN_FILENO);
            close(fd);

            execlp(command, command, NULL);

            perror("exec");
            exit(1);
        }

        wait(NULL);

        dup2(saved_stdin, STDIN_FILENO);
        close(saved_stdin);
        close(fd);

        printf("Input redirection completed successfully.\n");
        return 0;
}
```

OUTPUT:

```
ananya@Ananya: /mnt/d/OSSP_Programs/Skill/Week7$ nano input_redirect.c
ananya@Ananya: /mnt/d/OSSP_Programs/Skill/Week7$ gcc input_redirect.c -o input_redirect
ananya@Ananya: /mnt/d/OSSP_Programs/Skill/Week7$ nano input_redirect.c
ananya@Ananya: /mnt/d/OSSP_Programs/Skill/Week7$ ./input_redirect
Enter command with input redirection: cat < input.txt
Hello from input file!
Input redirection completed successfully.
ananya@Ananya: /mnt/d/OSSP_Programs/Skill/Week7$ |
```

OBSERVATION:

The program successfully demonstrated input redirection by parsing the < operator, opening the specified input file using open(), and redirecting stdin to the file using dup2(). The cat command was executed in a child process and correctly received the contents of input.txt as its input. The program also handled invalid or missing files using perror() and restored the original standard input after execution. Thus, file parsing, input stream redirection, process execution, error handling, and stream restoration were successfully demonstrated.

Task-2:

To Parse Output Redirection, Create Output Files, Redirect stdout, Handle Permissions, Verify Output Data, Test Redirection

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main()
{
    char input[200];

    printf("Enter command with output redirection: ");
    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    char *redirect = strchr(input, '>');

    if (redirect == NULL)
    {
        printf("No output redirection found.\n");
        return 1;
    }

    *redirect = '\0';

    char *command = input;
    char *filename = redirect + 1;

    while (*filename == ' ')
        filename++;

    while (*command == ' ')
        command++;

    char *end = command + strlen(command) - 1;
    while (end > command && *end == ' ')
    {
        *end = '\0';
        end--;
    }

    int fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);
```

```

if (fd == -1)
{
    perror("File open");
    return 1;
}

int saved_stdout = dup(STDOUT_FILENO);

if (fork() == 0)
{
    dup2(fd, STDOUT_FILENO);
    close(fd);

    execlp(command, command, NULL);

    perror("exec");
    exit(1);
}

wait(NULL);

dup2(saved_stdout, STDOUT_FILENO);
close(saved_stdout);
close(fd);

printf("Output redirection completed successfully.\n");

return 0;
}

```

OUTPUT:

```

ananya@Ananya: /mnt/d/OSSP_Programs/Skill/Week7$ nano output_redirect.c
ananya@Ananya: /mnt/d/OSSP_Programs/Skill/Week7$ gcc output_redirect.c -o output_redirect
ananya@Ananya: /mnt/d/OSSP_Programs/Skill/Week7$ ./output_redirect
Enter command with output redirection: ls > output.txt
Output redirection completed successfully.
ananya@Ananya: /mnt/d/OSSP_Programs/Skill/Week7$ cat output.txt
input.txt
input_redirect
input_redirect.c
output.txt
output_redirect
output_redirect.c
ananya@Ananya: /mnt/d/OSSP_Programs/Skill/Week7$ █

```

OBSERVATION:

The program successfully demonstrated output redirection by parsing the > operator and separating the command from the output filename. The output file was created/opened using open() with appropriate write and create permissions, and dup2() was used to redirect stdout from the terminal to the file. The command was executed in a child process, and the generated output was verified using cat. Error handling was also implemented for cases where the file could not be created or accessed, and the original stdout was restored after execution.

SESSION 14 :

Task - 1 :

To Parse Append Operator, Open Files in Append Mode, Redirect Streams, Preserve Existing Data, Validate Appends, Test Scenarios

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main()
{
    char input[200];

    printf("Enter command with append redirection: ");
    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    char *redirect = strstr(input, ">>");

    if (redirect == NULL)
    {
        printf("No append redirection found.\n");
        return 1;
    }

    *redirect = '\0';

    char *command = input;
    char *filename = redirect + 2;

    while (*filename == ' ')
        filename++;

    while (*command == ' ')
        command++;

    char *end = command + strlen(command) - 1;

    while (end > command && *end == ' ')
    {
        *end = '\0';
        end--;
    }

    int fd = open(filename, O_WRONLY | O_CREAT | O_APPEND, 0644);
```

```

    if (fd == -1)
    {
        perror("File open");
        return 1;
    }

    int saved_stdout = dup(STDOUT_FILENO);

    if (fork() == 0)
    {
        dup2(fd, STDOUT_FILENO);
        close(fd);

        char *args[20];
        int i = 0;

        char *token = strtok(command, " ");
        while (token != NULL && i < 19)
        {
            args[i++] = token;
            token = strtok(NULL, " ");
        }

        args[i] = NULL;
        execvp(args[0], args);

        perror("exec");
        exit(1);
    }

    wait(NULL);

    dup2(saved_stdout, STDOUT_FILENO);
    close(saved_stdout);
    close(fd);

    printf("Append redirection completed successfully.\n");

    return 0;
}

```

OUTPUT:

```

ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week7$ gcc append_redirect.c -o append_redirect
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week7$ ./append_redirect
Enter command with append redirection: echo First line >> output.txt
Append redirection completed successfully.
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week7$ cat output.txt
First line
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week7$ ./append_redirect
Enter command with append redirection: echo Second line >> output.txt
Append redirection completed successfully.
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week7$ cat output.txt
First line
Second line
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week7$ █

```

OBSERVATION:

The program successfully demonstrated append redirection using the >> operator. The existing contents of the output file were preserved, while new command output was added to the end of the file. The program used O_APPEND to open the file in append mode and dup2() to redirect stdout to the file. The command was executed successfully using a child process, and the resulting file contents were verified using cat. Thus, parsing the append operator, preserving existing data, redirecting streams, and validating the appended output were successfully demonstrated.

Task - 2:

To Parse stderr Redirection, Redirect Error Streams, Capture Error Output, Verify File Contents, Handle Failures, Test Error Cases

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main()
{
    char input[200];

    printf("Enter command with stderr redirection: ");
    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    char *redirect = strstr(input, "2>");

    if (redirect == NULL)
    {
        printf("No stderr redirection found.\n");
        return 1;
    }

    *redirect = '\0';

    char *command = input;
    char *filename = redirect + 2;

    while (*filename == ' ')
        filename++;

    while (*command == ' ')
        command++;

    char *end = command + strlen(command) - 1;

    while (end > command && *end == ' ')
    {
        *end = '\0';
        end--;
    }

    int fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);
```

```

if (fd == -1)
{
    perror("File open");
    return 1;
}

int saved_stderr = dup(STDERR_FILENO);

if (fork() == 0)
{
    dup2(fd, STDERR_FILENO);
    close(fd);

    char *args[20];
    int i = 0;

    char *token = strtok(command, " ");

    while (token != NULL && i < 19)
    {
        args[i++] = token;
        token = strtok(NULL, " ");
    }

    args[i] = NULL;

    execvp(args[0], args);

    perror("exec");
    exit(1);
}

wait(NULL);

dup2(saved_stderr, STDERR_FILENO);

close(saved_stderr);
close(fd);

printf("stderr redirection completed successfully.\n");

return 0;

```

OUTPUT:

```

ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week7$ nano stderr_redirect.c
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week7$ gcc stderr_redirect.c -o stderr_redirect
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week7$ ./stderr_redirect
Enter command with stderr redirection: ls nonexistent 2> error.txt
stderr redirection completed successfully.
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week7$ cat error.txt
ls: cannot access 'nonexistent': No such file or directory
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week7$ █

```

OBSERVATION:

The program successfully demonstrated stderr redirection using the `2>` operator. The error stream was redirected from the terminal to a specified file using `dup2()`. When an invalid command or non-existent file was used, the resulting error message was captured in `error.txt` and verified using `cat`. The program also handled file-opening failures and restored the original stderr after execution. Thus, error stream redirection, error capturing, verification, and failure handling were successfully demonstrated.